SHERLOCK

Security Review For DODO

Collaborative Audit Prepared For: **DODO**
Lead Security Expert(s): **0xadrii**
panprog
Date Audited: **December 23 - December 25, 2024**

Introduction

A leveraged market making solution to minimize IL and improve on liquidity management. The solution provides yield for retail LPs, higher profits for expert LPs, and better liquidity for traders.

Scope

Repository: DODOEX/Uniswap-swap-router-contracts

Audited Commit: e6f8603374e77d72b8fe84753b46f06177e4db52

Final Commit: e6f8603374e77d72b8fe84753b46f06177e4db52

Repository: DODOEX/Uniswap-v2-core

Audited Commit: fb13569358632a01f9eedb2a5feaf5ac5ef4236e

Final Commit: e3eee951aa7a42f65605a8990d8a2c419180afa8

Repository: DODOEX/Uniswap-v2-periphery

Audited Commit: 6fd1f8a6ba8db8cb6c62b4059c4b825489f63844

Final Commit: 74cf7086ac7ec146b7c0bb4d039dda21d258415f

Repository: DODOEX/Uniswap-v3-core

Audited Commit: 548bb8bb3afef20d1e40345f6795d8e5a8fdcfed

Final Commit: 244087f6fe360824ac3ff8a03e35af28c2ada35a

Findings

Each issue has an assigned severity:

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	2	6

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: Incorrect usage of original uniswap v2 interfaces instead of new ones with fees causes reverts in multiple functions

Source: https://github.com/sherlock-audit/2024-12-dodo/issues/63

Summary

The changes from the original uniswap v2 code includes addition of the fee field, so that pair is now identified by token0, token1 and fee. This is reflected in local interfaces and code changes, however, in multiple places interface from the original uniswap is still imported and used, identifying the pair only using token0 and token1 (without fee). This causes reverts in multiple places due to absence of these function signatures in the new contract code.

Vulnerability Detail

The following functions from uniswap v2 core and libraries are changed to include fee in the new code:

- `UniswapV2Factory.createPair`
- `UniswapV2Factory.getPair`
- `UniswapV2Library.pairFor`
- `UniswapV2Library.getReserves`
- `UniswapV2Library.getAmountOut`
- `UniswapV2Library.getAmountsOut`
- `UniswapV2Library.getAmountIn`
- `UniswapV2Library.getAmountsIn`

All these function take an additional `fee` argument compared to original uniswap v2 code.

The original uniswap v2 imported interface (without the `fee` argument) is still used here: https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-periphery/contracts/UniswapV2Router01.sol#L41-L42 https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-periphery/contracts/examples/ExampleSwapToPrice.sol#L45 https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78a

f77e335465b00a464a8/Uniswap-v2-periphery/contracts/examples/ExampleOracleSimple.sol#L28 https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-periphery/contracts/examples/ExampleSlidingWindowOracle.sol#L70 https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-periphery/contracts/examples/ExampleSlidingWindowOracle.sol#L108 https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-periphery/contracts/examples/ExampleFlashSwap.sol#L35 https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-periphery/contracts/examples/ExampleFlashSwap.sol#L51 https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-periphery/contracts/examples/ExampleFlashSwap.sol#L61 https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-swap-router-contracts/contracts/lens/MixedRouteQuoterV1.sol#L61 https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-swap-router-contracts/contracts/lens/TokenValidator.sol#L69

Impact

Some core library functions do not work and revert instead.

Code Snippet

Tool Used

Manual Review

Recommendation

Import interfaces from the local code and use correct function arguments (including fee) in the referenced code. Possibly to catch all these issues at compile time - remove all imports of the uniswap code, import only from the local interfaces.

Discussion

Skyewwww

Fix: https://github.com/DODOEX/Uniswap-v2-periphery/pull/1

We consider the contracts inside `/examples` and `/lens` won't be used, so we'd like to keep these files remain unaltered.

panprog

Router code: Fix confirmed. Examples and Lens code: acknowledged and won't be fixed.

Issue M-2: `computeProfitMaximizingTrade` uses fixed 0.3% fee instead of using variable specified fee.

Source: https://github.com/sherlock-audit/2024-12-dodo/issues/67

Summary

The `UniswapV2LiquidityMathLibrary.computeProfitMaximizingTrade` function uses fixed fee of 0.3%, disregarding that fee can now be different. As the function is used in some other functions, all of them calculate the trade sizes incorrectly.

Vulnerability Detail

This is the part of the function which uses 1000 and 997 values: https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-periphery/contracts/libraries/UniswapV2LiquidityMathLibrary.sol#L27-L34

This function is also used inside `getReservesAfterArbitrage`: https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-periphery/contracts/libraries/UniswapV2LiquidityMathLibrary.sol#L57

While `getReservesAfterArbitrage` has a `fee` parameter, and uses correct function to fetch pair reserves, the amounts calculation is incorrect since it doesn't use the `fee` provided, but uses fixed 0.3% fee instead.

Impact

Incorrect calculations of the arbitrage amounts library functions, potentially causing funds loss by anyone using the library functions to create arbitrage bots.

Code Snippet

Tool Used

Manual Review

Recommendation

Add `fee` argument to the `computeProfitMaximizingTrade` function and use it in calculations instead of fixed 0.3%

Discussion

Skyewwww

Fix: https://github.com/DODOEX/Uniswap-v2-periphery/pull/1

panprog

Fix confirmed

Issue L-1: Malicious `feeToSetter` can brick any pool to prevent liquidity providers removing liquidity by setting large `lpMtRatio` value

Source: https://github.com/sherlock-audit/2024-12-dodo/issues/64

The protocol has acknowledged this issue.

Summary

The uniswap v2 code includes the ability to set protocol fee, which is a percentage of all swap fees which go to `feeTo` address. The original uniswap v2 has this hardcoded as `1/6` of all swap fees, but in the new code `feeToSetter` address (admin) can set any value. The issue is that if this value is too high (`uint256.max` or close to it), the calculations will revert in the following line:

https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-core/contracts/UniswapV2Pair.sol#L124

This will cause all attempts to mint or burn pool shares to revert (deposit or withdraw liquidity).

Even though the `feeToSetter` is trusted admin, there is no reason to give it such power to brick any pool when it can easily be prevented.

Vulnerability Detail

Scenario when any pool can be bricked:

- `feeTo` is set to non-0 address
- `lpMtRatio` is set to a very large value (`uint256.max`)

Note: swaps will still work in the pool, only change of liquidity (mint or burn) will revert. Admin can still fix it at any time by setting a smaller `lpMtRatio` value.

Impact

Malicious admin can stop all deposits and withdrawals from any pool. Severity is Low, because this requires malicious admin. Still, Uniswap v2 is all about trustless operations and this issue creates too much unnecessary centralization risk.

Code Snippet

Tool Used

Manual Review

Recommendation

Limit `lpMtRatio` value to some sane number (1e6 or similar).

Issue L-2: `FeeRate` requirement is inconsistent in the `UniswapV2Factory` and `UniswapV2Pair`

Source: https://github.com/sherlock-audit/2024-12-dodo/issues/65

Summary

When the `FeeRate` is set, it's value is checked twice: in the `UniswapV2Factory`: https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-core/contracts/UniswapV2Factory.sol#L26

and in the `UniswapV2Pair`: https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-core/contracts/UniswapV2Pair.sol#L86

As seen from the code, the feeRate can be 0 (in the factory), but must be greater than 0 (in the pair). The value of `0` doesn't cause any issues, so it should probably be allowed. Also, it doesn't make sense to check the value twice, so one require can be removed.

Vulnerability Detail

Impact

`FeeRate` can not be 0 while it should probably be allowed, inconsistency in the require code.

Code Snippet

Tool Used

Manual Review

Recommendation

Remove require from the pair or make both of them the same.

Discussion

Skyewwww

Fix: https://github.com/DODOEX/Uniswap-v2-core/pull/1

panprog

Fix confirmed

Issue L-3: When protocol fee ratio is changed on a live pool, the new ratio is charged on pending swap fees instead of being applied only from the time of the change

Source: https://github.com/sherlock-audit/2024-12-dodo/issues/66

The protocol has acknowledged this issue.

Summary

The protocol fee is charged at the time of mint and burn, but the newly introduced function to change it (`setLpMtRatio`) can happen at any time and is applied immediately, meaning that all swap fees accumulated since the last mint/burn operation will have the new protocol fee ratio taken from these pending swap fees, which can unexpectedly reduce liquidity providers profits.

Vulnerability Detail

Example scenario:

- The protocol fee ratio (`lpMtRatio`) is at default (6)
- sqrt(lastK) = 10, sqrt(currentK) = 40 (due to swap fees), meaning a profit of 30 is pending. Any mint/burn operation will mint 5 pool shares (1/6 of profit) to `feeTo` address, meaning liquidity provider's pending profit is 24.
- Admin sets a new `lpMtRatio` to 1 (100% of swap fees)
- Now any burn/mint operation will mint 30 shares (100% of profit) to `feeTo` address, meaning liquidity provider's profit is 0 (they lose expected pending profit of 24 due to new admin setting).

Impact

New `lpMtRatio` is applied to pending profit instead of only being applied from the time of change. Since this is admin setting, admin can mint/burn dust amount before setting new ratio, or timing the change to not cause such issues. Still, this is a noisance which can be easily solved with the code, thus the severity is Low.

Code Snippet

Tool Used

Manual Review

Recommendation

Mint appropriate amount of pool shares to `feeTo` address and update `lastK` before setting `lpMtRatio` to a new value.

Issue L-4: `computeLiquidityValue` function uses fixed protocol fee ratio of `1/6` instead of the actual value from the pool

Source: https://github.com/sherlock-audit/2024-12-dodo/issues/68

Summary

The new code allows admin to change the default protocol fee ratio of `1/6` to any value. However, the helper library function `computeLiquidityValue` still uses fixed default ratio of `1/6`: https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-periphery/contracts/libraries/UniswapV2LiquidityMathLibrary.sol#L90

This function is also used in the `getLiquidityValue` function: https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v2-periphery/contracts/libraries/UniswapV2LiquidityMathLibrary.sol#L113

Vulnerability Detail

Impact

Incorrect calculation in library functions for live pools with protocol fee set to anything other than default.

Code Snippet

Tool Used

Manual Review

Recommendation

Fetch the correct ratio value from the pool's contract and use this instead of fixed value of 5 (`1/6` ratio).

Discussion

Skyewwww

Fix: https://github.com/DODOEX/Uniswap-v2-periphery/pull/1

panprog

Fix confirmed

Issue L-5: `UniswapV3Pool.setFeeProtocol` has incorrect upper limit on the protocol fee values

Source: https://github.com/sherlock-audit/2024-12-dodo/issues/69

Summary

`UniswapV3Pool.setFeeProtocol` function was modified to allow any values (in original uniswap it was only 0 or 4-10 range). Now it allows the values in the 0..16 range: https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v3-core/contracts/UniswapV3Pool.sol#L839

However, these protocolFee values are stored as 4-bit numbers: https://github.com/sherlock-audit/2024-12-dodo/blob/3b414c59a34b0aa0cd78af77e335465b00a464a8/Uniswap-v3-core/contracts/UniswapV3Pool.sol#L842-L843

This means that a value of 16 (which is a 5-bit value) messes things up (so admin setting a value of 16 will get no fee instead etc.). The correct upper limit is 15.

Vulnerability Detail

Impact

Incorrect protocol fee charged if admin tries to set allowed value of 16 for protocol fee. This is low severity, because it's an admin setting who should be aware on the actual correct values.

Code Snippet

Tool Used

Manual Review

Recommendation

Change the upper limit for protocol fee to 15.

Discussion

Skyewwww

Fix: https://github.com/DODOEX/Uniswap-v3-core/pull/1

panprog

Fix confirmed

Issue L-6: `FeeRate` value of 10000 (100%) doesn't seem to be usable

Source: https://github.com/sherlock-audit/2024-12-dodo/issues/70

The protocol has acknowledged this issue.

Summary

The pool's fee rate can be set to any value between 0 and 10000 (100%). However, the value of 10000 seems to be useless: no swap can be made with such value, only the donation. It might make sense to allow fee to be in the [0;10000) range.

Vulnerability Detail

Impact

A fee value of 10000 creates unusable pool (no swap can be made). No real issue with it, just a suggestion.

Code Snippet

Tool Used

Manual Review

Recommendation

Do not allow fee rate to be 10000.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.